



Estrutura de Dados não Lineares

Prof. Thiago Koman Schmidt



Planejamento da aula

- O que são **árvores** em estrutura de dados
- **Definições:** raiz, folhas, altura, nível, profundidade
- Representação por **ponteiros e vetores**

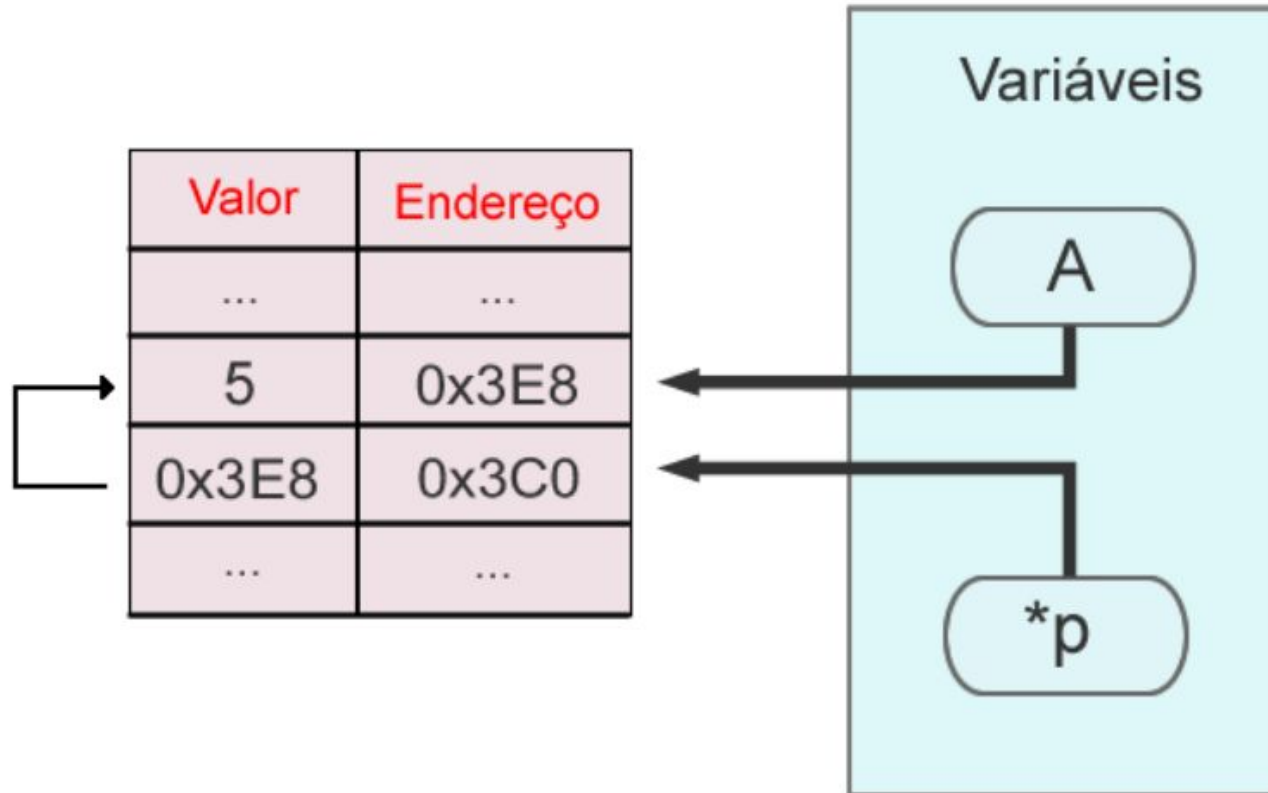


Ponteiro

O que é um ponteiro?

- É uma variável que contém um endereço de memória
- Esse endereço é normalmente a posição de uma outra variável na memória
- Se uma variável contém o endereço de uma outra, então diz-se que a primeira variável “aponta” para a segunda

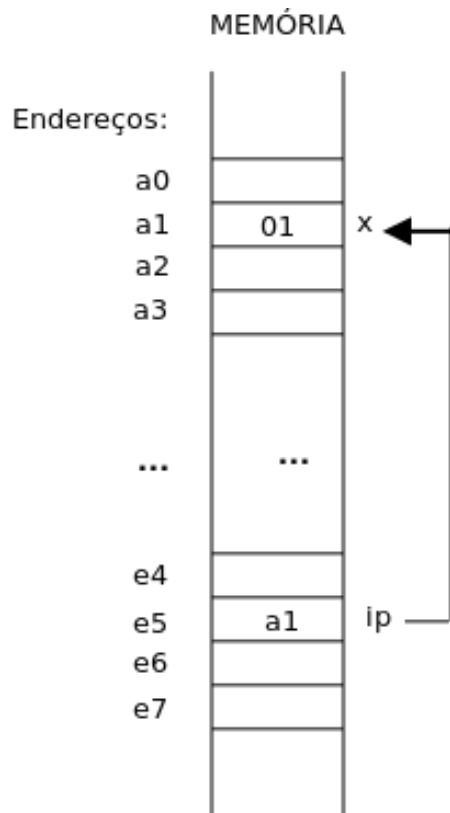
O que é um ponteiro?



O que é um ponteiro?

- A declaração de um ponteiro deve informar
 - Tipo para o qual ele aponta (int, char, float, etc)
 - “*” (asterisco)
 - Nome do ponteiro
 - Declaração: tipo * nome_variavel;
- Exemplos:
 - • char * nome;
 - • int * px;
 - • float * preco;
- O tipo base do ponteiro define que tipo de variáveis o ponteiro pode apontar
- Toda a aritmética de ponteiros é feita por meio do tipo base.

O que é um ponteiro?



```
int x;    //x é uma variável inteiro
int *ip;  //ip é um ponteiro para inteiro
x = 1;    //x agora vale 1
ip=&x;    //ip agora aponta para x
```

Operadores de Ponteiros

- Operador de endereço &
- Operador unário *

&

*

Operador de endereço &

- Retorna o endereço de uma variável
- O endereço de uma variável não tem relação nenhuma com o seu conteúdo
- Deve atuar apenas em operandos associados a um único endereço, como variáveis comuns ou elementos individuais de uma matriz
- Deve ser pensado como “o endereço de”

Operador de endereço &

```
int main(int argc, char *argv[])
{
    int x = 3;
    int * p;
    p = &x;
    printf("Valor do ponteiro: %d\n", p);
    printf("Endereco de x: %d", &x);
    return 0;
}
```

```
Valor do ponteiro: 10485316
Endereco de x: 10485316
```

Operador unário *

- É o complemento do operador &
- Retorna o valor da variável localizada no endereço que o segue
- Pode atuar apenas em operandos que representam endereços
- Deve ser pensado como “no endereço”

Operador unário *

```
int main(int argc, char *argv[])
{
    int x = 3;
    int * p;
    p = &x;
    printf("Valor da variavel no endereco apontado: %d\n", *p);
    printf("Valor de x: %d", x);
    return 0;
}
```

```
Valor da variavel no endereco apontado: 3
Valor de x: 3
```



Structs

O que é uma Struct?

- Agrupamento de variáveis sob um mesmo nome
- Representa entidades do mundo real (ex.: Aluno, Ponto, Produto)
- Similar a uma classe, mas por padrão os membros são públicos

Sintaxe Básica

```
struct Aluno {  
    std::string nome;  
    int idade;  
    float nota;  
};
```



```
Aluno a1;  
a1.nome = "Maria";  
a1.idade = 20;  
a1.nota = 9.5;
```

Inicialização de Structs

- Inicialização direta dos campos
- Pode ser usada em listas ou vetores de structs

```
Aluno a2 = {"João", 22, 8.7};
```

Structs com Funções

- Structs também podem ter funções membros

```
struct Ponto {  
    int x, y;  
    void mostrar() {  
        cout << "(" << x << ", " << y << ")\n";  
    }  
};
```

Structs vs Classes

- Struct: membros públicos por padrão
- Classe: membros privados por padrão
- Ambos podem ter:
 - Construtores
 - Métodos
 - Herança (menos comum em structs)

Exemplo completo

```
#include <iostream>
using namespace std;

struct Produto {
    string nome;
    double preco;
};

int main() {
    Produto p1 = {"Notebook", 3500.00};
    cout << "Produto: " << p1.nome
         << " - R$" << p1.preco << endl;
}
```

Quando usamos Structs?

- ❖ Modelar entidades simples
 - ❖ Agrupar dados relacionados
 - ❖ Substituir variáveis soltas
-
- Se precisar de encapsulamento avançado → prefira classes



Árvores

Introdução à Estrutura de Dados em Árvore

Uma árvore é uma estrutura de dados hierárquica.

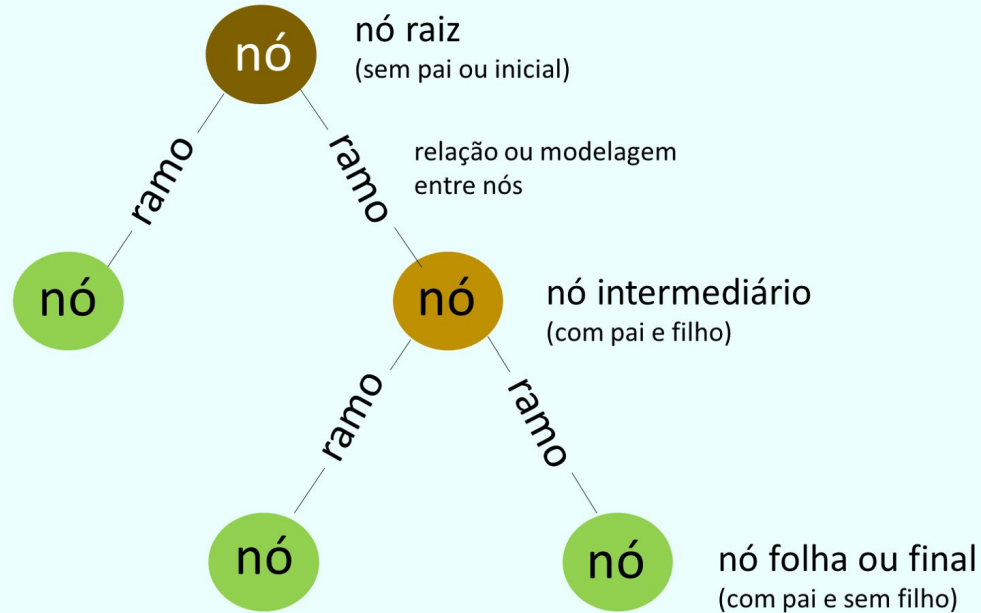
Constitui-se de nós conectados por arestas.

Utilizada para representar relações hierárquicas como:

- Diretórios de arquivos
- Famílias (genealogia)
- Expressões matemáticas

Exemplo de aplicações: árvores de decisão, parsers de código, IA (minimax), bancos de dados (indexação).

Estrutura em Árvore



Elementos Fundamentais

Raiz (root): nó principal da árvore, onde tudo começa.

Nó (node): unidade fundamental da árvore. Pode conter valor e referências para filhos.

Filho (child): um nó descendente de outro.

Pai (parent): nó acima de um filho.

Folha (leaf): nó sem filhos.

Subárvore: qualquer nó com seus descendentes.

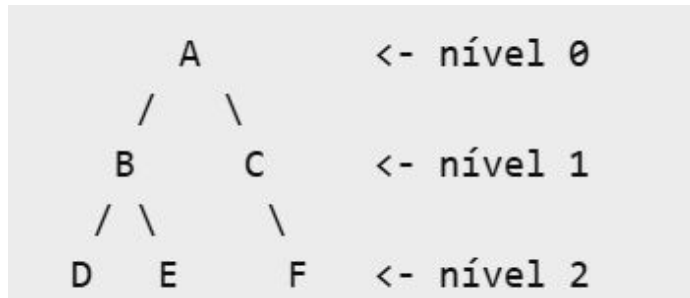
Altura, Nível e Profundidade

Altura de um nó: maior distância até uma folha (descendo).

Altura da árvore: altura da raiz.

Nível de um nó: quantos passos até a raiz (subindo).

Profundidade: sinônimo de nível, mas vista do ponto de vista da raiz até o nó.





Representações de árvores

Representação com Ponteiros

Muito comum em árvores binárias

- Cada nó aponta para seus filhos

```
struct Node {  
    int valor;  
    Node* esquerdo;  
    Node* direito;  
};
```

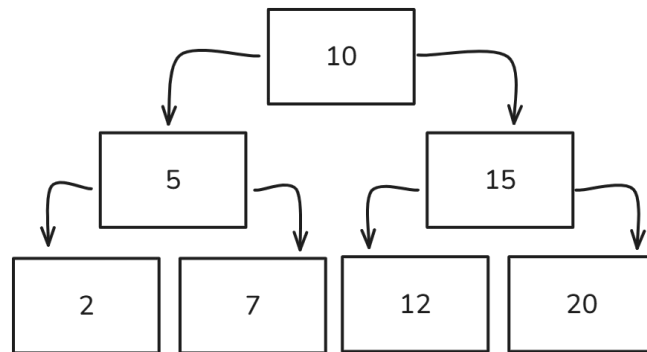
```
Node* raiz = new Node{10, nullptr, nullptr};  
raiz->esquerdo = new Node{5, nullptr, nullptr};  
raiz->direito = new Node{15, nullptr, nullptr};
```

Representação com Vetores

Representa uma árvore binária completa em vetor:

Fórmula:

- Pai no índice i
- Filho esquerdo: $2*i + 1$
- Filho direito: $2*i + 2$

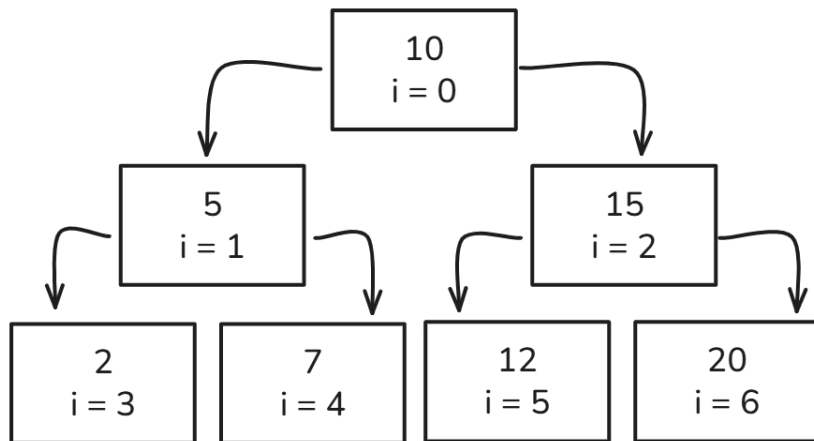


```
vector<int> árvore = {10, 5, 15, 2, 7, 12, 20};
```

****Espaço desperdiçado se não for completa**

Representação com Vetores

```
vector<int> árvore = {10, 5, 15, 2, 7, 12, 20};
```



Fórmula:

- Pai no índice i
- Filho esquerdo: $2*i + 1$
- Filho direito: $2*i + 2$

Quando usar cada representação?

Estrutura	Vantagens	Desvantagens
Ponteiros	Flexível, ideal para qualquer forma	Gasta mais espaço (endereços)
Vetor (array)	Acesso rápido, simples para completas	Ineficiente para árvores esparsas



Atividade

Atividade

Objetivo: Criar uma árvore binária simples com três níveis (ou seja, a raiz, seus filhos e os filhos dos filhos).

Parte 1 – Representação com ponteiros:

Implemente a árvore em C++ usando estrutura de nós com ponteiros. A árvore deve conter pelo menos 7 nós (completa até o terceiro nível).

Parte 2 – Representação com vetor:

Represente a mesma árvore utilizando um vetor, respeitando as posições conforme a fórmula:

- Filho esquerdo de $i \rightarrow 2*i + 1$
- Filho direito de $i \rightarrow 2*i + 2$

Desafio Bônus

*Implemente um percurso em ordem e imprima os valores dos nós da esquerda para a direita.

DÚVIDAS?

